Name - Ravindra Gour

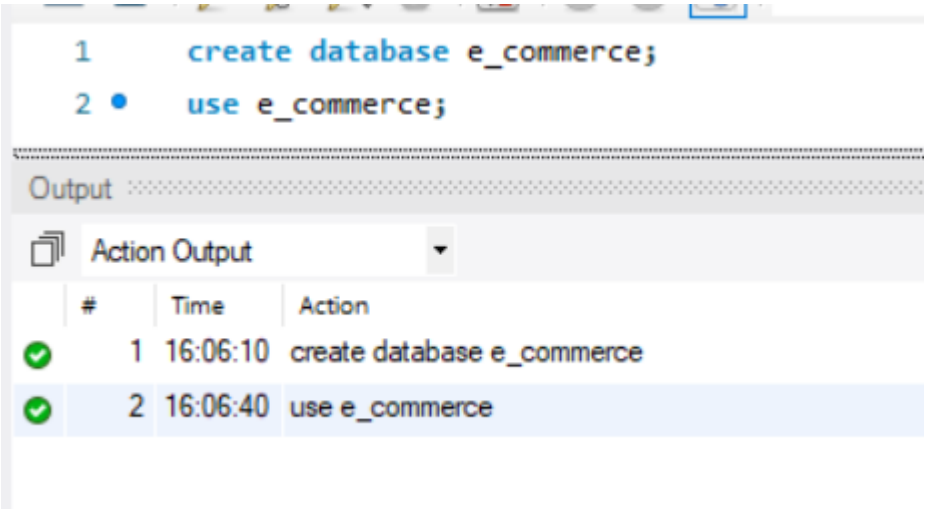
Create Database:

```
create database e_commerce;
```

Exp: using the command **create database <name_of_db>** we can create the database.

```
use e_commerce;
```

Exp: use <name_of _db> command selects DB to work on.



Creating Tables:

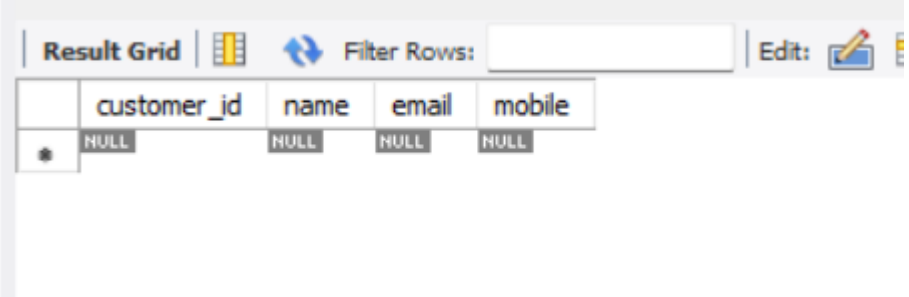
Customers:

```
create table Customers (
   customer_id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50),
   email VARCHAR(50),
   mobile VARCHAR(15)
);
```

Exp:

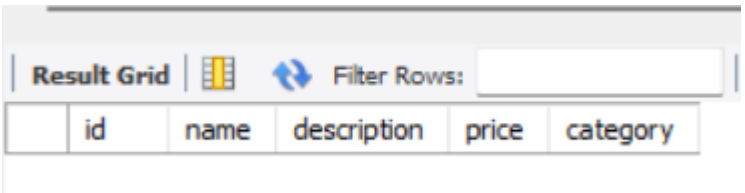
- **Create table <table_name>** command used to create a new table in the database
- **customer_id , name , email , mobile** are the column name
- **INT** and **VARCHAR** are the data type or **varchar(length_of_string).**
- **AUTO_INCREMEN** automatically increases value (1, 2, 3, ...)

- **PRIMARY KEY** uniquely identifies each row (no duplicates, no NULL)



Products:

```
create table Products (
    id INT,
    name VARCHAR(50) NOT NULL,
    description VARCHAR(200),
    price DECIMAL(10,2) NOT NULL,
    category VARCHAR(50)
);
```



Modify Tables (Using ALTER):

Exp: ALTER is used to change **structure**, not data an **existing table.**

Syntax: **ALTER TABLE <table_name> <operation>**;

Operations:

ADD: Used to **add a new column** to an existing table.

MODIFY: Used to **change datatype, size, or constraints.**

DROP: Used to **remove a column** completely.

RENAME: Used to **rename column** or table.

A. Add not null on name and email in the Customers table

```
ALTER TABLE Customers MODIFY name VARCHAR(50) NOT NULL;

ALTER TABLE Customers MODIFY email VARCHAR(50) NOT NULL;
```

10	18:38:41	ALTER TABLE Customers MODIFY name VARCHAR(50) NOT NULL
11	18:38:47	ALTER TABLE Customers MODIFY email VARCHAR(50) NOT NULL

B. Add a unique key on email in the Customers table.

```
ALTER TABLE Customers ADD UNIQUE (email);
```

12	18:43:14	ALTER TABLE Customers ADD UNIQUE (email)

C. Add column age in the Customers table.

```
ALTER TABLE Customers ADD age INT;
```

13	18:45:02	ALTER TABLE Customers ADD age INT

D. Change column name from id to product_id in the Products table;

```
ALTER TABLE Products CHANGE id product_id INT;
```

14	18:48:45	ALTER TABLE Products CHANGE id product_id INT

E. Add primary key and auto increment on product_id in the Products table

```
ALTER TABLE Products MODIFY product_id INT
AUTO_INCREMENT PRIMARY KEY;
```

15	18:49:49	ALTER TABLE Products MODIFY product_id INT AUTO_INCREMENT PRIMARY KEY

F. Change datatype of description from varchar to text in the Products table.

```
ALTER TABLE Products MODIFY description TEXT;
```

16	18:54:39	ALTER TABLE Products MODIFY description TEXT

Create Order table:

```
CREATE TABLE Orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    quantity INT NOT NULL,
    order_date DATE NOT NULL,
    status ENUM('Pending','Success','Cancel'),
    payment_method ENUM('Credit','Debit','UPI'),
    total_amount DECIMAL(10,2) NOT NULL,
```

```
    FOREIGN KEY (customer_id) REFERENCES
Customers(customer_id)
);
```

	order_id	customer_id	product_id	quantity	order_date	status	payment_method	total_amount
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Modify Orders Table(using Alter keyword):

A. Change table name Order -> Orders.

```
 RENAME TABLE Orders TO Orders;
```

In this case MYsql takes only name orders not order so this query does not run in mysql because the table name is Orders already.

The reason is that: Order is a mysql keyword, not a normal word.

B. Set default value pending in status.

```
ALTER TABLE Orders MODIFY status
ENUM('Pending','Success','Cancel') DEFAULT 'Pending';
```

23 19:14:51 ALTER TABLE Orders MODIFY status ENUM('Pending','Success','Cancel') DEFAULT 'Pending'

C. Modify payment_method ENUM to add one more value: 'COD'.

```
ALTER TABLE Orders MODIFY payment_method
ENUM('Credit','Debit','UPI','COD');
```

24 19:16:32 ALTER TABLE Orders MODIFY payment_method ENUM('Credit','Debit','UPI','COD')

D. Make product id as foreign key.

```
ALTER TABLE Orders ADD FOREIGN KEY (product_id)
REFERENCES Products(product_id);
```

25 19:17:41 ALTER TABLE Orders ADD FOREIGN KEY (product_id) REFERENCES Products(product_id)

Insert 20 sample records in all the tables

Customers:

customer_id	name	email	mobile	age
1	Ravi Kumar	ravi1@gmail.com	9000000001	22
2	Aman Singh	aman2@gmail.com	9000000002	23
3	Priya Sharma	priya3@gmail.com	9000000003	21
4	Neha Verma	neha4@gmail.com	9000000004	24
5	Rahul Das	rahul5@gmail.com	9000000005	25
6	Sneha Gupta	sneha6@gmail.com	9000000006	22
7	Vikas Yadav	vikas7@gmail.com	9000000007	26
8	Pooja Mishra	pooja8@gmail.com	9000000008	23
9	Ankit Jain	ankit9@gmail.com	9000000009	27
10	Karan Patel	karan10@gmail.com	9000000010	24
11	Rohit Mehta	rohit11@gmail.com	9000000011	22
12	Simran Kaur	simran12@gmail.com	9000000012	23
13	Deepak Sha...	deepak13@gmail.com	9000000013	28
14	Nikita Roy	nikita14@gmail.com	9000000014	21
15	Suresh Reddy	suresh15@gmail.com	9000000015	29
16	Meena Joshi	meena16@gmail.com	9000000016	26
17	Ajay Thakur	ajay17@gmail.com	9000000017	27
18	Divya Kapoor	divya18@gmail.com	9000000018	22
19	Arjun Nair	arjun19@gmail.com	9000000019	25
20	Kavita Iyer	kavita20@gmail.com	9000000020	24
NULL	NULL	NULL	NULL	NULL

Products:

product_id	name	description	price	category
1	Laptop	Gaming Laptop	800.00	Electronics
2	Smartphone	Android Phone	300.00	Electronics
3	Headphones	Wireless Headphones	100.00	Electronics
4	Keyboard	Mechanical Keyboard	70.00	Electronics
5	Mouse	Gaming Mouse	40.00	Electronics
6	Monitor	24 inch Monitor	200.00	Electronics
7	Tablet	Android Tablet	250.00	Electronics
8	Charger	Fast Charger	25.00	Electronics
9	Camera	DSLR Camera	500.00	Electronics
10	Speaker	Bluetooth Speaker	120.00	Electronics
11	T-shirt	Cotton T-shirt	20.00	Clothing
12	Jeans	Denim Jeans	40.00	Clothing
13	Jacket	Winter Jacket	80.00	Clothing
14	Shoes	Sports Shoes	60.00	Clothing
15	Watch	Analog Watch	90.00	Accessories
16	Bag	Travel Bag	70.00	Accessories
17	Perfume	Body Perfume	50.00	Accessories
18	Book	Programming Book	30.00	Books
19	Notebook	Stationery Notebook	10.00	Books
20	Pen	Ball Pen Pack	5.00	Books

Orders:

order_id	customer_id	product_id	quantity	order_date	status	payment_method	total_amount
1	1	1	1	2025-03-01	Success	UPI	800.00
2	2	2	2	2025-03-02	Pending	COD	600.00
3	3	3	1	2025-03-03	Cancel	Debit	100.00
4	4	4	3	2025-03-04	Success	Credit	210.00
5	5	5	2	2025-03-05	Pending	UPI	80.00
6	6	6	1	2025-03-06	Success	Debit	200.00
7	7	7	2	2025-03-07	Success	Credit	500.00
8	8	8	4	2025-03-08	Pending	COD	100.00
9	9	9	1	2025-03-09	Success	UPI	500.00
10	10	10	2	2025-03-10	Cancel	Debit	240.00
11	11	11	3	2025-03-11	Success	Credit	60.00
12	12	12	2	2025-03-12	Pending	UPI	80.00
13	13	13	1	2025-03-13	Success	COD	80.00
14	14	14	2	2025-03-14	Success	Debit	120.00
15	15	15	1	2025-03-15	Pending	Credit	90.00
16	16	16	2	2025-03-16	Success	UPI	140.00
17	17	17	1	2025-03-17	Cancel	Debit	50.00
18	18	18	3	2025-03-18	Success	Credit	90.00
19	19	19	5	2025-03-19	Pending	COD	50.00
20	20	20	10	2025-03-20	Success	UPI	50.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Perform following queries:

a. Count the number of products as product_count in each category.

```
SELECT category, COUNT(*) AS product_count
FROM Products
GROUP BY category;
```

EXP: In this query, we use the aggregate function COUNT() along with GROUP BY. The GROUP BY clause groups all rows based on the category column, and then COUNT(*) calculates the number of products in each category.

category	product_count
Electronics	10
Clothing	4
Accessories	3
Books	3

B. Retrieve all products that belong to the 'Electronics' category, have a price between $50 and $500, and whose name contains the letter 'a'.

```
SELECT *
FROM Products
WHERE category = 'Electronics'
AND price BETWEEN 50 AND 500
AND name LIKE '%a%';
```

EXP: In this query, we are selecting all columns from the Products table. Then we apply conditions using WHERE. First, we filter only those products whose category is Electronics. Then we check the price should be between 50 and 500. After that, we filter those products whose name contains the letter ‘a’. Finally, only those products which satisfy all these conditions will be shown.

product_id	name	description	price	category
2	Smartphone	Android Phone	300.00	Electronics
3	Headphones	Wireless Headphones	100.00	Electronics
4	Keyboard	Mechanical Keyboard	70.00	Electronics
7	Tablet	Android Tablet	250.00	Electronics
9	Camera	DSLR Camera	500.00	Electronics
10	Speaker	Bluetooth Speaker	120.00	Electronics
NULL	NULL	NULL	NULL	NULL

C. Get the top 5 most expensive products in the 'Electronics' category, skipping the first 2.

```
SELECT *
FROM Products
WHERE category = 'Electronics'
ORDER BY price DESC
LIMIT 5 OFFSET 2;
```

EXP: In this query, we are selecting all columns from the Products table. First, we filter only those products whose category is Electronics. Then we sort those products in descending order of price, meaning highest price first. After that, we skip the first 2 records using OFFSET and then display the next 5 records using LIMIT.

product_id	name	description	price	category
2	Smartphone	Android Phone	300.00	Electronics
7	Tablet	Android Tablet	250.00	Electronics
6	Monitor	24 inch Monitor	200.00	Electronics
10	Speaker	Bluetooth Speaker	120.00	Electronics
3	Headphones	Wireless Headphones	100.00	Electronics
NULL	NULL	NULL	NULL	NULL

D. Retrieve customers who have not placed any orders.

```
SELECT *
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

EXP: In this query, we are selecting all data from Customers and joining it with Orders using LEFT JOIN based on customer_id.

LEFT JOIN means all customers will be included, even if they don't have any orders. Then we apply a condition where order customer_id is NULL, which means those customers who don't have any matching orders. So finally, we get customers who have not placed any orders.

customer_id	name	email	mobile	age	order_id	customer_id	product_id	quantity	order_date	status	payment_method	total_amount

E. Find the average total amount spent by each customer.

```
SELECT customer_id, AVG(total_amount) AS avg_spent
FROM Orders
GROUP BY customer_id;
```

EXP: In this query, we are selecting customer_id from Orders table. Then we group all the orders based on customer_id. After grouping, we calculate the average of total_amount for each customer using the AVG function. Finally, we show the average spend of each customer.

customer_id	avg_spent
1	800.000000
2	600.000000
3	100.000000
4	210.000000
5	80.000000
6	200.000000
7	500.000000
8	100.000000
9	500.000000
10	240.000000
11	60.000000
12	80.000000
13	80.000000
14	120.000000
15	90.000000
16	140.000000
17	50.000000
18	90.000000
19	50.000000
20	50.000000

F. Get the products that have a price less than the average price of all products.

```
SELECT *
FROM Products
WHERE price < (SELECT AVG(price) FROM Products);
```

EXP: In this query, we are selecting all products from the Products table. Inside the WHERE condition, we use a subquery to calculate the average price of all products. Then we compare each product price with that average price, and only those products whose price is less than the average price will be shown.

product_id	name	description	price	category
3	Headphones	Wireless Headphones	100.00	Electronics
4	Keyboard	Mechanical Keyboard	70.00	Electronics
5	Mouse	Gaming Mouse	40.00	Electronics
8	Charger	Fast Charger	25.00	Electronics
10	Speaker	Bluetooth Speaker	120.00	Electronics
11	T-shirt	Cotton T-shirt	20.00	Clothing
12	Jeans	Denim Jeans	40.00	Clothing
13	Jacket	Winter Jacket	80.00	Clothing
14	Shoes	Sports Shoes	60.00	Clothing
15	Watch	Analog Watch	90.00	Accessories
16	Bag	Travel Bag	70.00	Accessories
17	Perfume	Body Perfume	50.00	Accessories
18	Book	Programming Book	30.00	Books
19	Notebook	Stationery Notebook	10.00	Books
20	Pen	Ball Pen Pack	5.00	Books
NULL	NULL	NULL	NULL	NULL

G. Calculate the total quantity of products ordered by each customer.

```
SELECT customer_id, SUM(quantity) AS total_quantity
FROM Orders
GROUP BY customer_id;
```

EXP: In this query, we are selecting customer_id from the Orders table. Then we group all the orders based on customer_id. After grouping, we calculate the total quantity of products for each customer using SUM function. Finally, we display the total quantity for each customer.

customer_id	total_quantity
1	1
2	2
3	1
4	3
5	2
6	1
7	2
8	4
9	1
10	2
11	3
12	2
13	1
14	2
15	1
16	2
17	1
18	3
19	5
20	10

H. List all orders along with customer name and product

name.SELECT o.order_id, c.name AS customer_name, p.name AS product_name
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Products p ON o.product_id = p.product_id;

EXP: In this query, we are selecting order_id from Orders table, customer name from Customers table, and product name from Products table. First, we join Orders with Customers using customer_id, then we join Orders with Products using product_id. So finally, we get order details along with customer name and product name.

order_id	customer_name	product_name
1	Ravi Kumar	Laptop
2	Aman Singh	Smartphone
3	Priya Sharma	Headphones
4	Neha Verma	Keyboard
5	Rahul Das	Mouse
6	Sneha Gupta	Monitor
7	Vikas Yadav	Tablet
8	Pooja Mishra	Charger
9	Ankit Jain	Camera
10	Karan Patel	Speaker
11	Rohit Mehta	T-shirt
12	Simran Kaur	Jeans
13	Deepak Sharma	Jacket
14	Nikita Roy	Shoes
15	Suresh Reddy	Watch
16	Meena Joshi	Bag
17	Ajay Thakur	Perfume
18	Divya Kapoor	Book
19	Arjun Nair	Notebook
20	Kavita Iyer	Pen

I. Find products that have never been ordered.

```
SELECT *
FROM Products p
LEFT JOIN Orders o ON p.product_id = o.product_id
WHERE o.product_id IS NULL;
```

EXP: In this query, we are selecting all products from the Products table and joining it with Orders using LEFT JOIN based on product_id. LEFT JOIN means all products will be shown even if they don't have any orders. Then we apply a condition where order product_id is NULL, which means those products that are not present in Orders. So finally, we get products that have not been ordered.

product_id	name	description	price	category	order_id	customer_id	product_id	quantity	order_date	status	payment_method	total_amount